HACKATHON EDIMA

Plan du volet langue

Duala MVP · validation native interne, glossaire et modèle open-source

Version
2.0 — pivot duala

Date
9 novembre 2026

Périmètre
Traductions duala, glossaire, phrases-modèles

Contrainte
Modèle gratuit, validation par l'équipe + 2 relecteurs natifs

Résumé exécutif

- **Langue MVP : duala (`dua`).** L'ewondo (`ewo`) devient secondaire : il ne revient que si le temps le permet.
- **Changement de situation :** un locuteur natif est dans l'équipe et deux relecteurs natifs sont disponibles. Le blocage linguistique est levé ; le chemin critique devient la **traduction puis la validation du glossaire**, pas le fine-tune.
- **Approche « manuel d'abord » :** les 72 termes et 24 phrases-modèles sont traduits à la main en duala, puis validés. Le modèle open-source sert d'**aide au brouillon** pour le texte inédit (F9), jamais à marquer `valide`.
- **Données :** 15 351 paires anglais–duala (MT560, CC-BY-4.0), Bible duala alignable, BOUQuET `dua` pour l'évaluation, Duala TTS pour l'audio, NTeALan et dictionnaires pour le lexique.
- **Validation :** toi (locuteur natif) + 2 relecteurs. Aucun terme n'est affiché avant `dua_status = valide` ; repli français sinon.
- **Matériel :** GTX 1650 Ti 4 Go insuffisante pour NLLB-600M. Colab/Kaggle T4 recommandé.
- **Audio (D2) :** reporté après le texte. La voix française reste le fallback.

> **Politique de licences (contrainte projet) :** on n'utilise que des licences **gratuites et non copyleft**. Sont acceptées : domaine public/CC0, CC-BY, MIT, Apache-2.0, BSD. Sont écartées : licences payantes/propriétaires et licences à partage à l'identique (GPL, AGPL, LGPL, MPL, CC BY-SA, CC BY-NC-SA, ODbL…), car elles imposeraient d'ouvrir le code ou les données d'Edima. Les licences *NonCommercial* (CC BY-NC) et NOODL sont traitées comme **conditionnelles** : OK pour la démo, à revoir avant tout usage commercial.

1. Décisions actées

Décision	Choix	Conséquence
Langue MVP	Duala	Ewondo reporté ; le code garde les deux langues mais priorise `dua`.
Validation	Toi + 2 relecteurs natifs	Plus de recrutement bloquant ; validation interne planifiée.
Périmètre	Glossaire 72 → 150 termes ; 24 → 50 phrases-modèles	Pas de MT générale pour le MVP.
Méthode	Traduction manuelle d'abord, modèle en brouillon	La qualité ne dépend pas du fine-tune.
Modèle	Fine-tune d'une base permissive : **M2M100-418M (MIT)** en priorité	Pas d'API payante, pas de copyleft ; NLLB-600M seulement en référence qualité (CC BY-NC).
Audio	Reporté après le texte	Fallback `speechSynthesis` français en attendant.
Partage	Dossier modèle/adapter + JSON + métriques	Le collaborateur n'a pas besoin de réentraîner.

2. Contraintes matérielles et partage

2.1 Ce que la machine locale permet

Modèle	Fine-tune local 4 Go	Remarque
NLLB-200-distilled-600M	**Non**	Poids + optimiseur + activations > 8 Go.
M2M100-418M (MIT)	**QLoRA 4-bit, limite**	Base permissive recommandée ; lent en local, confortable sur T4.
MarianMT 74M	**Oui**	Pas de duala natif : utile seulement comme brique pivot.
Colab / Kaggle T4 16 Go	**Oui**	Gratuit, ne nécessite pas un PC puissant.

Machine constatée : Intel i5 10e génération, GTX 1650 Ti 4 Go, Python 3.14. Pour PyTorch, utiliser Colab/Kaggle ou un venv Python 3.10–3.12.

2.2 Comment partager le résultat de l'entraînement

L'entraînement produit un **dossier de modèle**, pas seulement des chiffres :

- `config.json`, `model.safetensors` (ou `pytorch_model.bin`), `tokenizer.json`, `generation_config.json` ;
- si LoRA : `adapter_config.json` + `adapter_model.safetensors` (quelques Mo) ;
- métriques : `metrics.json`, `predictions.jsonl`, logs.

Tu zippes ce dossier et tu l'envoies (Google Drive, WeTransfer, clé USB, ou HF Hub gratuit). Le collaborateur le charge avec :

```
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

model = AutoModelForSeq2SeqLM.from_pretrained("chemin/vers/dossier")
tokenizer = AutoTokenizer.from_pretrained("chemin/vers/dossier")
```

Avec LoRA, il télécharge le modèle de base puis applique l'adapter (PEFT) : c'est le plus léger à partager. Pour Edima, le modèle n'est pas nécessaire côté extension : on **pré-génère** les traductions validées dans `terms.json` / `templates.json`.

3. Ressources vérifiées

3.1 Duala (prioritaire)

Ressource	Contenu	Licence	Statut	Usage
`michsethowusu/english-duala_sentence-pairs_mt560`	15 351 paires EN–DUA (textes JW.org, via MT560/OPUS)	CC-BY-4.0	**OK** (attribution)	Entraînement EN↔DUA / pivot
Bible duala — NT 1901 (archive.org)	Nouveau Testament duala, domaine public	Domaine public	**OK**	Alignement FR–DUA
Bible duala — *Betileda ba Bosangi* 1966/1970	Portions/Genèse, Rosetta Project	À vérifier	Conditionnel	Alignement (si licence claire)
Bible duala — Duala 2006 Edition	Bible complète (Alliance biblique du Cameroun)	Sous copyright	**À écarter**	Ne pas intégrer
`facebook/bouquet` — `dua_Latn`	504 dev + 854 test	CC-BY-4.0, gated	**OK** (attribution)	Évaluation uniquement
Glot500 — modèle `glot500-base`	Modèle 0,4B couvrant `dua_Latn`	Apache-2.0	**OK**	LM / score de fluidité
Glot500 — données	Corpus monolingue `dua_Latn`	Packaging CC0, sources sous licences variées	Conditionnel	Vérifier la source avant usage
Duala-TTS-Dataset (MDC)	1 521 enregistrements, 4 h 30, transcriptions AGLC	NOODL-1.0	Conditionnel	Audio (phase ultérieure)
NTeALan Duala–Français	191 entrées	CC BY-NC-SA 3.0	**À écarter** (SA)	Consultation seule, pas d'intégration
`dictionnaireduala.com` (P. Mbappé Bessémé)	Dictionnaire duálá–français en ligne	Droits réservés	**À écarter**	Consultation seule
Helmlinger 1972 (archive.org)	*Dictionnaire duala-français*	Sous copyright	**À écarter**	Consultation seule
Omnilingual ASR (Meta)	`dua_Latn`	Apache-2.0	**OK**	ASR / contrôle

NOODL-1.0 : gratuit et commercialisable pour les entités situées dans un pays en développement (dont le Cameroun) ; les entités de pays à revenu élevé doivent un accord de partage de bénéfices. Ce n'est pas du copyleft, mais ce n'est pas « libre sans condition » : à garder pour l'audio, pas pour le cœur du produit.

3.2 Ewondo (secondaire)

Ressource	Contenu	Licence	Usage
Ewondo-French Parallel Corpus (MDC)	3 631 paires	NOODL-1.0	Conditionnel (si reprise de l'ewondo)
`ELRs/Ewondo_Bible`	16 785 versets	CC-BY-4.0	OK (attribution)
`yemen2016/ewondo-french-model`	MarianMT 74,7M	Apache-2.0	OK

> **Point critique vérifié :** NLLB-200 **ne contient pas** le duala (ni le token `dua_Latn`, ni l'ewondo). M2M100, MMS-TTS et Google Translate ne le couvrent pas non plus. Tout fine-tune NLLB exige donc d'**ajouter** `dua_Latn` au tokenizer et d'initialiser son embedding (warm-start depuis des langues bantoues proches).

4. Stratégie de données (duala)

1. **Corpus principal :** MT560 EN–DUA (15 351 paires). Directement exploitable pour EN↔DUA. Pour FR↔DUA, deux voies : (a) fine-tune pivot via l'anglais ; (b) génération de paires FR–DUA synthétiques via un modèle EN→FR de qualité, puis filtrage (COMET, LaBSE, ratio de longueur) et relecture d'un échantillon. Le pivot introduit du bruit : à ne pas confondre avec de la donnée validée.
2. **Alignement Bible :** Bible duala + Bible Segond 1910 (domaine public). Potentiel de plusieurs dizaines de milliers de versets, c'est le principal levier pour FR↔DUA. Vérifier la versification et la licence de l'édition (privilégier le NT 1901 pour l'entraînement, éviter la 2006 sous copyright).
3. **BOUQuET `dua` :** dev/test pour l'évaluation externe. Accepter les conditions HF ; ne pas entraîner dessus.
4. **Lexiques :** NTeALan, dictionnaire duálá, Helmlinger — pour le glossaire et l'orthographe, pas pour la MT.
5. **Monolingue :** Glot500 `dua_Latn`, pour un LM de contrôle ou un score de fluidité.
6. **Audio :** Duala TTS (NOODL), à traiter après le texte.

4.1 Procédure d'alignement Bible duala

- Télécharger la Bible duala et la Segond 1910 (domaine public).
- Vérifier si les lignes duala sont des versets dans l'ordre canonique (Genèse 1, Jean 3:16, etc.).
- Si oui : alignement par index + filtrage (ratio de longueur, doublons, langue).
- Si non : alignement automatique (LaBSE + `vecalign`) + vérification manuelle de 50 paires.
- Sortie : `bible_parallel_dua.jsonl`. Si plus de 10 % de paires douteuses, on jette le lot.

Risques : versification/canon, orthographe et tons (AGLC), licence de la traduction biblique sous-jacente.

4.2 Nettoyage commun

- Normalisation Unicode NFC, conservation des tons (AGLC).
- Déduplication exacte et quasi-exacte.
- Filtre de longueur et ratio source/cible (0,3–3,0).
- Contrôle de langue (GlotLID) pour éviter le français ou l'anglais résiduel.
- Split 80/10/10 sans fuite entre train, validation et test.

4.3 Note sur l'ewondo

Le corpus Ewondo–Français (3 631 paires) et la Bible ewondo (16 785 versets) restent disponibles. On ne les traite qu'après la livraison du duala, et uniquement si le temps le permet.

5. Modèle

5.1 Baseline

- Ni M2M100 ni NLLB ne connaissent le duala : la vraie baseline est le modèle **après ajout du token** `dua_Latn`, ou un modèle EN↔DUA entraîné sur MT560.
- Métriques : chrF++ et BLEU (sacrebleu), plus analyse qualitative sur le vocabulaire bancaire.

5.2 Fine-tune

- **Base prioritaire :** `facebook/m2m100_418M` — licence **MIT**, compatible avec un produit fermé et commercial.
- **Référence qualité (optionnelle) :** `facebook/nllb-200-distilled-600M` — meilleur potentiel mais CC BY-NC-4.0, donc réservé à la démo/au non commercial.
- **Token :** ajouter `dua_Latn` au tokenizer + extension SentencePiece, warm-start des embeddings depuis des langues bantoues présentes (lingala, kikongo, luganda, kinyarwanda, swahili…).

- **Direction :** FR→DUA prioritaire (interface) ; DUA→FR secondaire.
- **Méthode :** LoRA ou full fine-tune sur Colab/Kaggle T4.
- **Repli local :** pas de MT duala fiable sur 4 Go. Le repli réaliste est **concordance + glossaire + validation humaine**.
- **Adaptation domaine :** 2e fine-tune sur le glossaire/phrases une fois brouillonnés et validés.

5.3 Évaluation et go/no-go

- BOUQuET `dua` dev + split interne.
- Comparaison baseline vs fine-tuné, et contrôle humain sur 50 phrases bancaires.
- **Go** si chrF++ du fine-tuné dépasse la baseline d'au moins 5 points et que la relecture juge les sorties utilisables comme brouillons ; sinon on reste sur glossaire + phrases-modèles + validation humaine.

> **Licences :** M2M100 est MIT (OK produit fermé/commercial) ; NLLB est CC-BY-NC-4.0 (démo hackathon OK, commercial à éviter). MT560 et BOUQuET sont CC-BY-4.0 (attribution). La Bible 2006 est sous copyright, le NT 1901 est dans le domaine public. On n'intègre aucune ressource CC BY-SA / CC BY-NC-SA (NTeALan, dictionnaire duálá, Helmlinger) : consultation seulement.

6. Workflow glossaire et phrases-modèles

1. **Export CSV :** `scripts/glossary-export.ts` produit un tableur traducteur (`id, fr, contexte, dua, statut_dua, notes`).
2. **Traduction interne :** toi (locuteur natif) remplis les 72 termes et 24 phrases en duala → statut `a_valider` .
3. **Relecture :** les 2 relecteurs natifs corrigent orthographe, tons et terminologie bancaire.
4. **Validation :** passage à `valide` uniquement après accord des relecteurs.
5. **Brouillons modèle :** utilisés pour les termes difficiles et pour F9, jamais publiés sans validation.
6. **Import :** `scripts/glossary-import.ts` valide le CSV avec Zod et vérifie les invariants.

6.1 Invariants d'import

- `dua_status = valide` ⇒ traduction duala non nulle.
- Placeholders `{ … }` préservés à l'identique.
- Identifiants uniques, catégories connues, pas de doublons.
- Statut par langue (`ewo_status` / `dua_status`) pour livrer le duala sans attendre l'ewondo.

7. Modifications de code

Fichier	Modification
`packages/shared/src/schemas.ts`	Remplacer le statut global par `ewo_status` et `dua_status` (ou une carte de statuts par langue).
`packages/shared/src/glossary.ts:35`	`fillTemplate` lit `template.dua` si `dua_status === "valide"` .
`packages/shared/src/glossary.ts:66`	`translateText` utilise le statut de la langue active (duala par défaut).
`packages/shared/src/glossary.test.ts`	Mettre à jour le test qui exige `a_traduire` + `null` partout ; ajouter un cas duala validé.
`packages/shared/src/numbers.ts:85`	Laisser `null` jusqu'à la grammaire des nombres duala validée.
`apps/server/src/index.ts`	Remplacer l'appel LLM payant par un sidecar local (FastAPI/ONNX) ou un fallback français.
`apps/extension`	Langue par défaut `dua` ; l'extension lit les JSON validés.

8. Outillage à ajouter

`ml/` — Python, hors workspace pnpm

- `requirements.txt` (transformers, datasets, sacrebleu, peft, accelerate).
- `notebooks/01-baseline.ipynb`
- `notebooks/02-finetune-nllb-dua.ipynb`
- `notebooks/03-eval-dua.ipynb`
- `scripts/fetch_data.py`
- `scripts/prepare_data.py`
- `scripts/align_bible_dua.py`
- `scripts/concordance.py`

`scripts/` — TypeScript

- `glossary-export.ts`
- `glossary-import.ts`
- `eval-glossary.ts`

`.gitignore`

- `ml/data/`
- `ml/models/`
- `ml/.venv/`

9. Intégration

- **MVP :** pré-générer les traductions duala validées dans `terms.json` / `templates.json`. Aucune inférence en production.
- **F9 (texte inédit) :** sidecar Python local ou ONNX Runtime appelé par Hono, sinon repli français. Pas d'API payante.
- **Audio :** pré-générer les fichiers avec le TTS duala ou attendre les enregistrements natifs. Reporté.
- **Partage collaborateur :** dossier modèle/adapter + `metrics.json` + `predictions.jsonl` + JSON validés.

10. Jalons

Jalon	Livrable	Critère de sortie
J1	Export CSV + traduction duala des 72 termes et 24 phrases par toi	Toutes les lignes en `a_valider`, placeholders intacts
J2	Relecture par 2 natifs	Accord des relecteurs, corrections intégrées
J3	Code branché (`dua_status`) + JSON validés	Tests passent, l'extension affiche le duala validé
J4	Données + baseline + test d'alignement Bible duala	Corpus et baseline reproductibles
J5	Fine-tune + évaluation BOUQuET `dua`	chrF++ > baseline + 5 points, sinon repli glossaire
J6	Éval 200 phrases / 100 montants + rétro-traduction	0 erreur montants/frais/avertissements, ≥ 90 % compris

11. Risques

Risque	Impact	Mitigation
Bruit du corpus pivot	FR–DUA synthétique imparfait	Filtrage COMET/LaBSE + relecture d'un échantillon ; ne pas utiliser comme donnée validée
Bible 2006 sous copyright	Usage juridique incertain	Privilégier le NT 1901 (domaine public) pour l'entraînement
Licence copyleft / NC	Obligation d'ouvrir le code ou restriction commerciale	Base M2M100 (MIT) ; écarter GPL/AGPL/CC BY-SA/CC BY-NC-SA ; NLLB seulement en démo
Dépendances npm copyleft	Contamination de la licence du produit	Auditer avec `license-checker` ; préférer MIT/Apache/BSD
NLLB sans token duala	Fine-tune non trivial	Ajout de token + warm-start bantou ; évaluer le gain réel
Alignement Bible incertain	Corpus bruité	Vérification manuelle, filtre de qualité, abandon si > 10 % douteux
Domaine bancaire absent	Modèle faible sur le métier	2e fine-tune sur glossaire/phrases + validation native
4 Go de VRAM	Pas de NLLB local	Colab/Kaggle T4 ; sinon pas de MT locale
Hallucinations	Traductions inventées	Statut `valide` obligatoire avant affichage
Orthographe / tons (AGLC)	Incohérences	Guide de style duala + relecture native

12. Prochaines étapes

1. Exporter le glossaire et les phrases-modèles en CSV (`glossary-export.ts`).
2. Remplir les traductions duala (toi) → `a_valider` .
3. Planifier la relecture avec les 2 natifs (orthographe, tons, termes bancaires).
4. Adapter le code au statut par langue et passer la langue par défaut à `dua` .
5. Télécharger MT560 EN–DUA, accepter BOUQuET, tester l'alignement Bible duala (NT 1901).
6. Auditer les licences : données retenues (OK uniquement) et dépendances npm (pas de GPL/AGPL).

7. Construire `ml/`, exécuter la baseline, puis fine-tune M2M100 (MIT) sur Colab/Kaggle.
8. Décider si l'audio D2 reste en fallback français pour la démo.

> **Question ouverte :** l'audio D2 est-il nécessaire pour la démo du hackathon, ou peut-il rester en fallback français jusqu'à la validation du texte ?
